IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

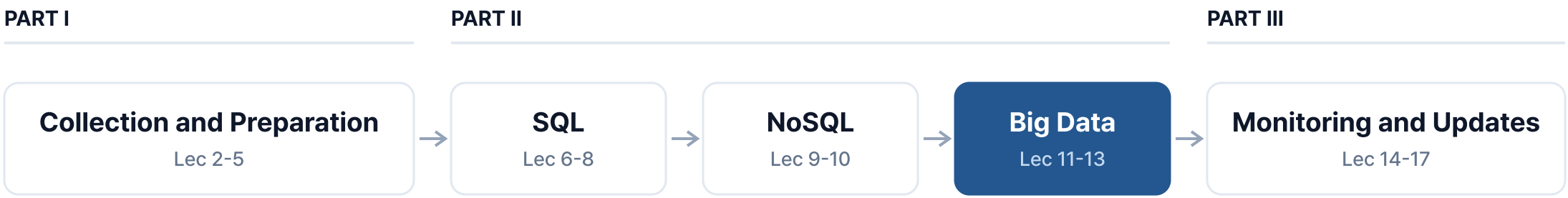
11.

Big Data I: Batch Processing

Sangwoo Mo · POSTECH

Industrial & Management Engineering

Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 11-13
Monitoring and Updates
Lec 14-17

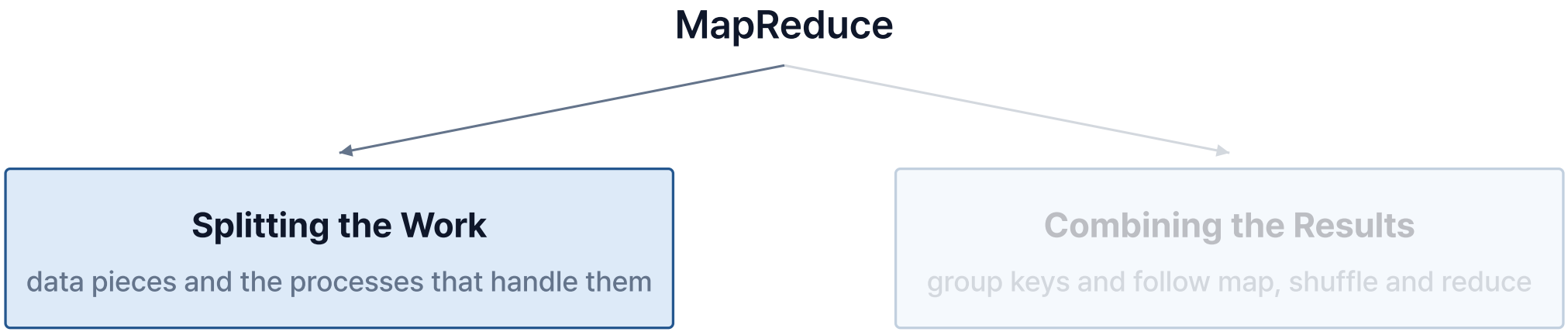
Outline: Big Data I

01	**MapReduce**	split data, map records and combine by key
02	**Spark**	describe table operations and run distributed jobs

01 · MapReduce

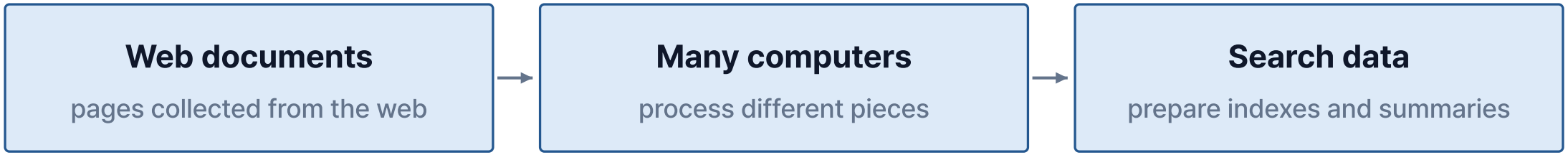
01	**MapReduce**	split data, map records and combine by key
02	**Spark**	describe table operations and run distributed jobs

§ Splitting the Work



Google Had a Data Problem

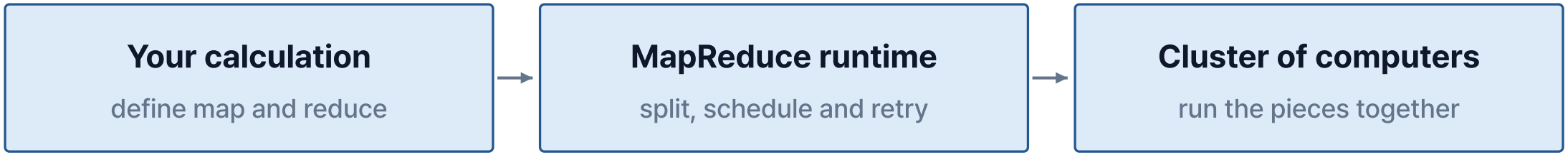
Google needed to turn large collections of web pages and request logs into useful data



Simple calculations became difficult when the input needed many computers

Google Publishes MapReduce

In 2004, Jeffrey Dean and Sanjay Ghemawat at Google published the MapReduce paper



Programmers describe the calculation; the runtime coordinates the computers

Hadoop Makes the Idea Available

Doug Cutting and Mike Cafarella developed Hadoop from the Nutch search project



Hadoop brought distributed storage and MapReduce to an open-source community

The Name Hadoop

A family nickname became the name of a distributed computing project

Hadoop was the name of a toy elephant

- Doug Cutting’s son gave that name to his elephant toy
- Cutting reused the name for the software project
- The name came from his family, rather than a technical abbreviation

One Sensor Reading

Each row records one temperature measurement from one machine

id	machine	temp
3	M1	85



Reading number

measurement 3

Machine name

M1 = one machine

Temperature

85 degrees Celsius

Row 3 says that machine M1 was measured at 85°C

Filtering Hot Readings

For one day of readings, keep every measurement at 80°C or above

Nine measurements from one day

id	machine	temp
1	M1	70
2	M2	90
3	M1	85
4	M1	95
5	M3	75
6	M2	60
7	M3	88
8	M1	82
9	M2	78



Five measurements at 80°C or above

id	machine	temp
2	M2	90
3	M1	85
4	M1	95
7	M3	88
8	M1	82

Keep a row if temp >= 80

The condition keeps five of the nine readings

Counting by Machine

Put the same machine name together, then count the readings in each group

M1 readings

id	temp
3	85
4	95
8	82

3 hot readings

M2 readings

id	temp
2	90

1 hot reading

M3 readings

id	temp
7	88

1 hot reading

Group by machine = put the same machine name together

M1 has three hot readings; M2 and M3 each have one

Why Many Machines

Distributed processing divides work across computers when one computer is insufficient

One computer reads everything

All input rows

one computer does all the work

Three computers share the reading

Computer 1: one piece

Computer 2: one piece

Computer 3: one piece

Same question, same answer; different computers handle different rows

Different computers read different rows to answer the same question

Scale Up or Scale Out

Scale up adds resources to one machine; scale out adds machines

Scale up

One larger machine

more memory / processing power

Simpler, until one machine is not enough

Scale out

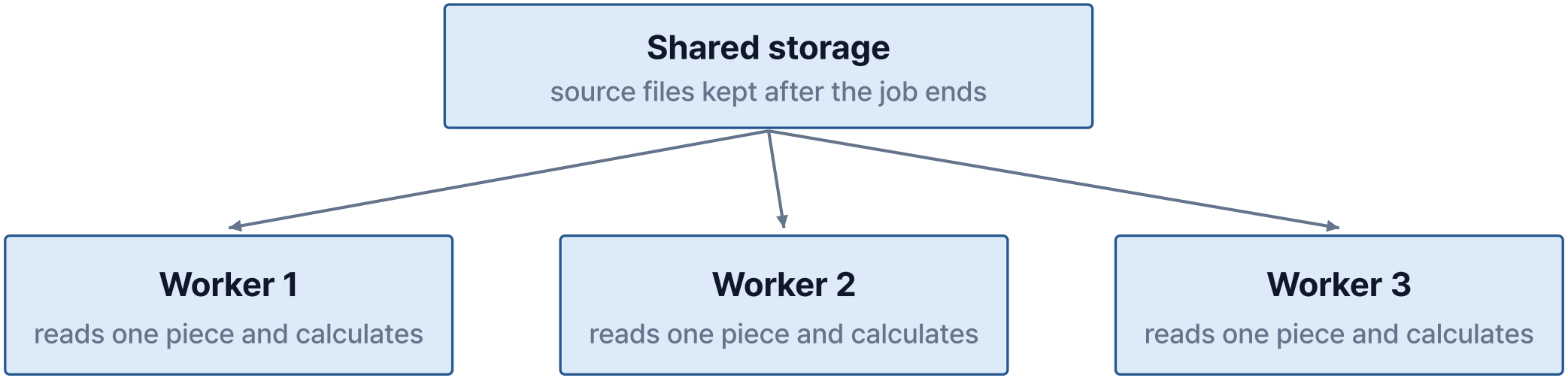
Worker 1 **Worker 2** **Worker 3**

More capacity, plus coordination and transfers

Extra machines add capacity and coordination work

Storage and Compute

Storage keeps the files; workers are processes that read files and calculate results



Storage keeps source data; workers run the computation

Durable input and temporary computation have different jobs

Input Partitions

A partition is one piece of data that can be processed independently

Partition 1

id	machine	temp
1	M1	70
2	M2	90
3	M1	85

3 rows

Partition 2

id	machine	temp
4	M1	95
5	M3	75
6	M2	60

3 rows

Partition 3

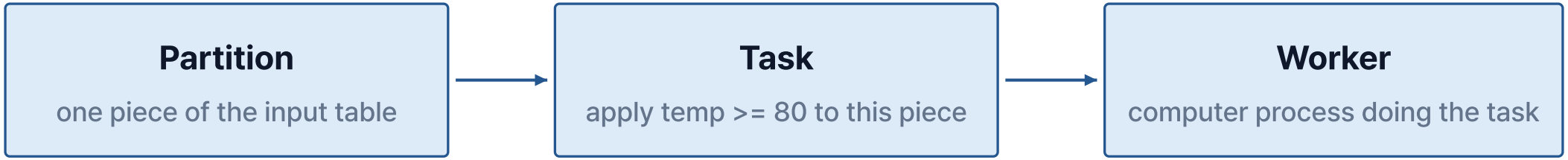
id	machine	temp
7	M3	88
8	M1	82
9	M2	78

3 rows

M1 appears in all three input partitions

Partitions, Tasks and Workers

The data piece, the computation and the process doing it have different names



A worker can process another partition after finishing this task

A task applies the filter to one partition on a worker

Local Filtering

Each computer can keep its hot readings without checking the other pieces

Partition 1

id	machine	temp
2	M2	90
3	M1	85

2 rows kept

Partition 2

id	machine	temp
4	M1	95

1 rows kept

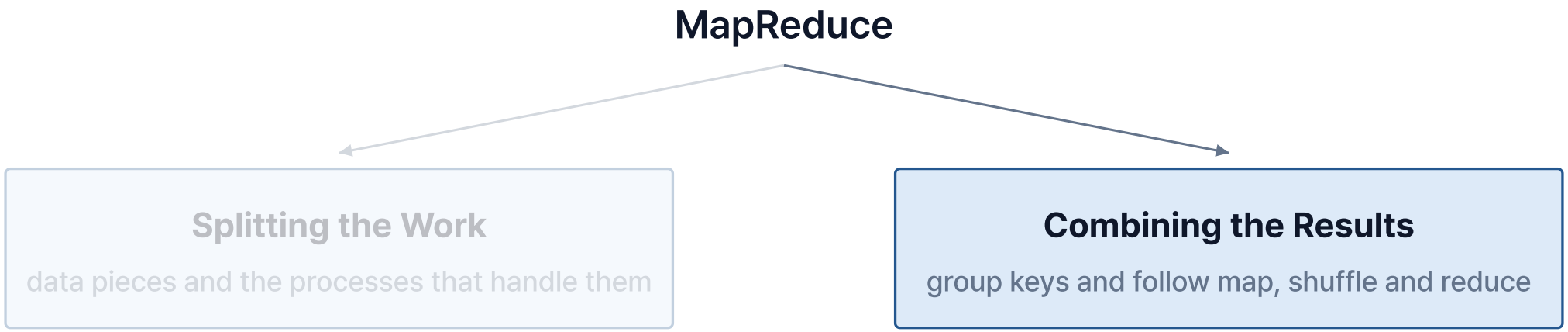
Partition 3

id	machine	temp
7	M3	88
8	M1	82

2 rows kept

Two, one and two rows remain in the three partitions

§ Combining the Results



Local Counts Are Not the Final Answer

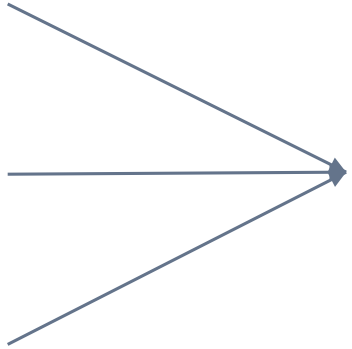
A global group must combine information from different partitions

Local answers

P1: M1=1, M2=1

P2: M1=1

P3: M1=1, M3=1



Global answer

machine	count
M1	3
M2	1
M3	1

M1: 1 + 1 + 1 = 3

Three local M1 counts must become one global count

The Grouping Key

A key is the value used to decide which records belong in the same group

id	machine	temp
3	M1	85
4	M1	95
8	M1	82

Key = machine name

M1 groups these three readings

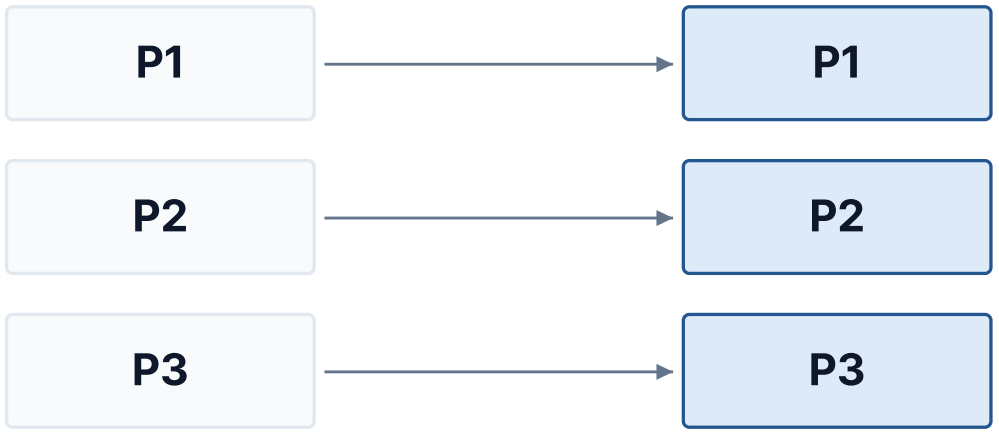
The reading IDs differ; the machine key is the same

Different reading IDs share the same machine key, M1

Shuffle

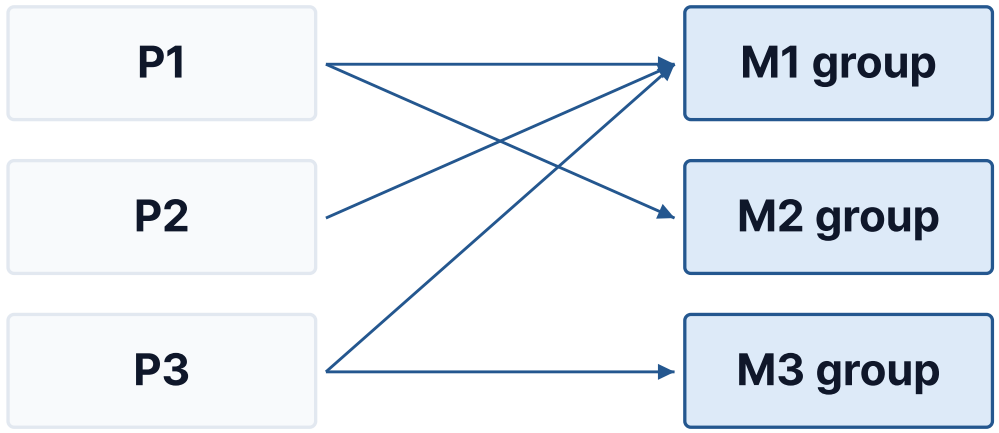
A shuffle redistributes data across partitions for a key-based operation

Filter within each partition



No cross-partition exchange

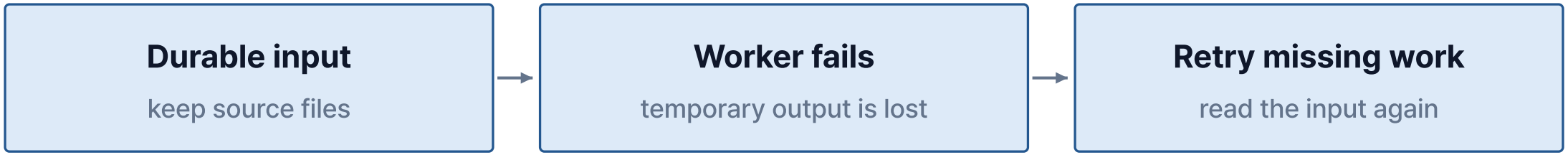
Group the same key together



Shuffle: records cross the network

Filtering stays local; a global grouping may move data

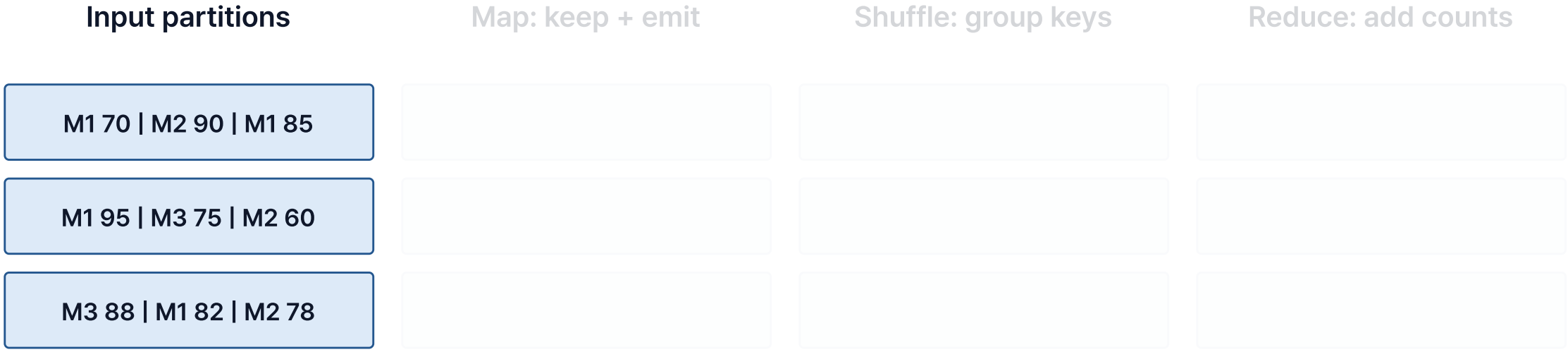
A Worker Can Fail



Retry needs available source data and a repeatable computation

The MapReduce Pattern

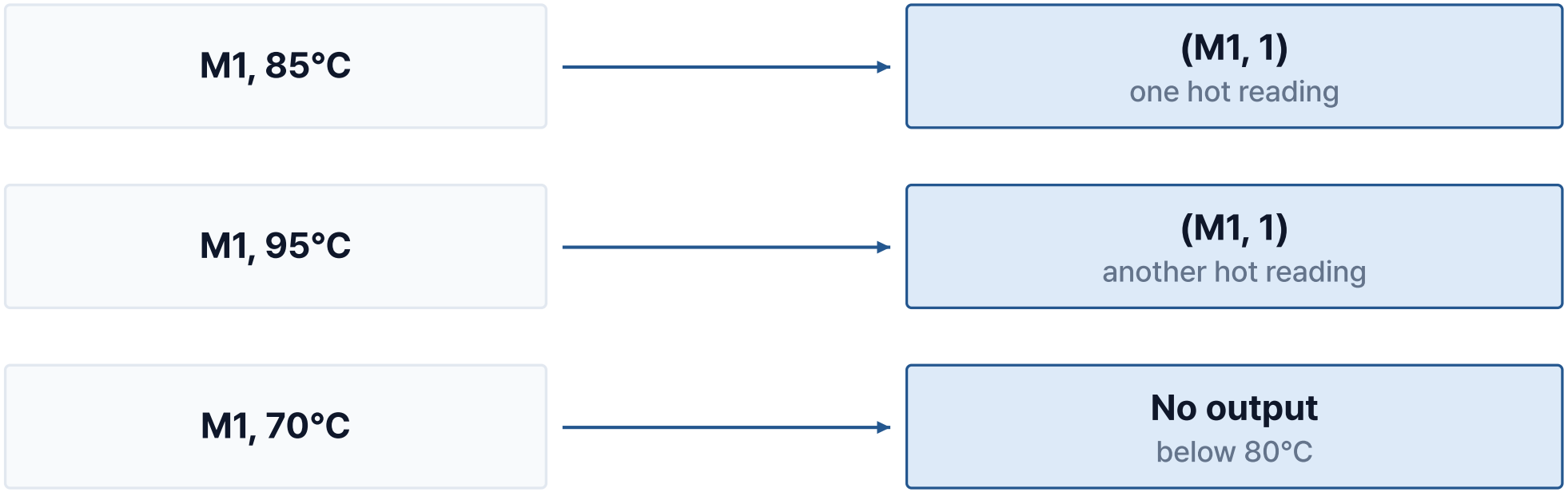
MapReduce organizes this job as per-row work, key grouping and per-group work



Follow the same nine readings through map, shuffle and reduce

One Reading, One Count

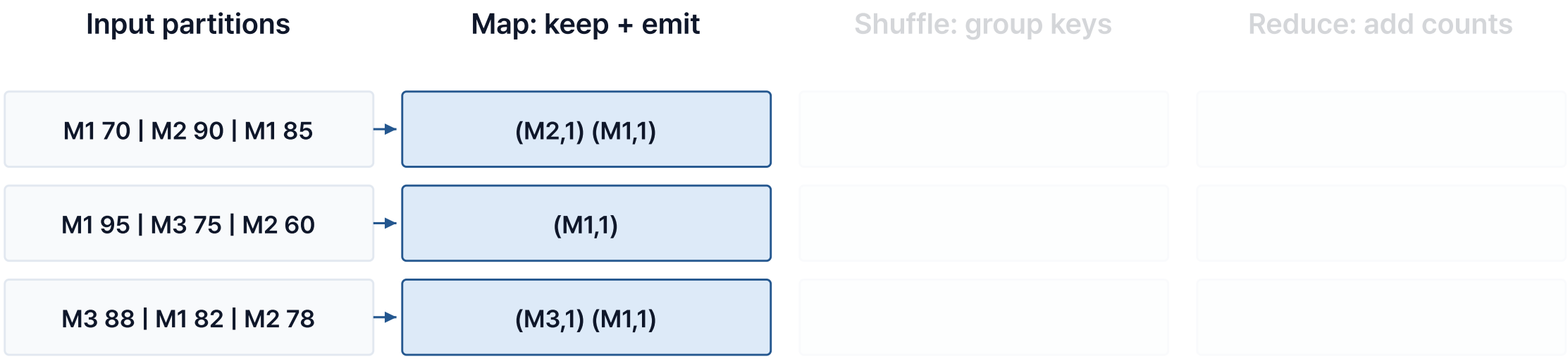
In this MapReduce example, map turns each hot reading into (machine, 1)



The machine names the group; 1 counts one reading, regardless of its temperature

Map: Keep and Emit

For each row at 80°C or above, map outputs (machine, 1); other rows produce nothing

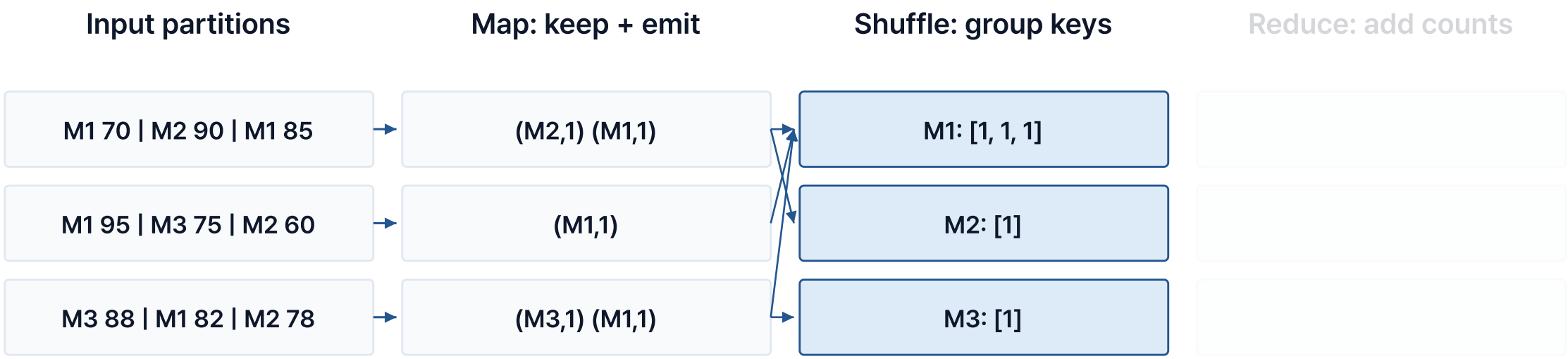


Selection: temp >= 80 | Key: machine | Value: 1

The value 1 represents one retained reading

Shuffle: Bring the Same Key Together

Every value for M1 meets in the same key group



Selection: temp >= 80 | Key: machine | Value: 1

The shuffle crosses the original input-partition boundaries

Reduce: Add Within Each Key

Reduce turns the values of each key into one result



Selection: temp >= 80 | Key: machine | Value: 1

Five readings become three grouped counts

Demo: Follow the Grouped Count

Browser simulation: predict the next step before revealing it

Back | Step | Reset

Input partitions	Map: keep + emit	Shuffle: group keys	Reduce: add
M1 70 | M2 90 | M1 85			
M1 95 | M3 75 | M2 60			
M3 88 | M1 82 | M2 78			

Input only: predict which readings pass temp >= 80 and how many remain.

Track M1 from three input partitions to one result

Answer: Follow the Grouped Count

Map emits five pairs, shuffle forms three keys, reduce returns three counts



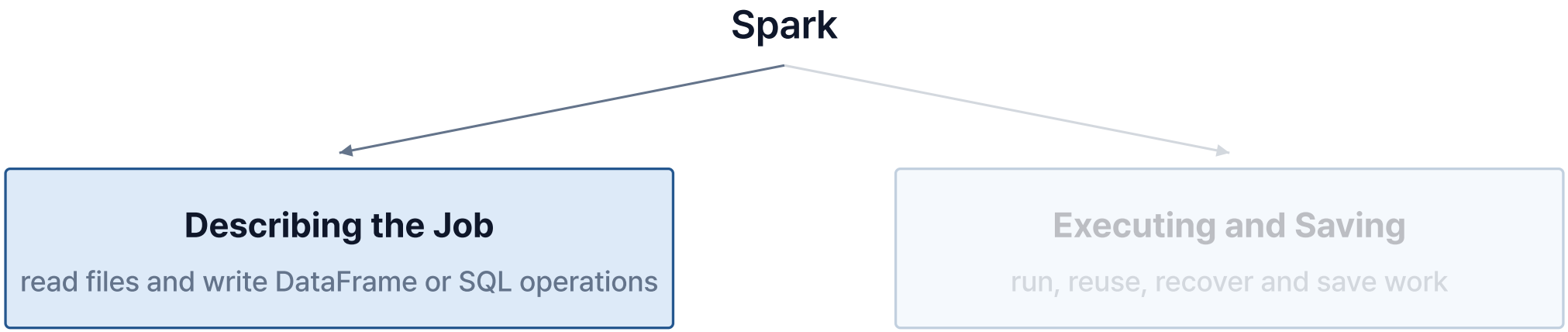
Selection: temp >= 80 | Key: machine | Value: 1

The output counts sum to five retained readings

02 · Spark

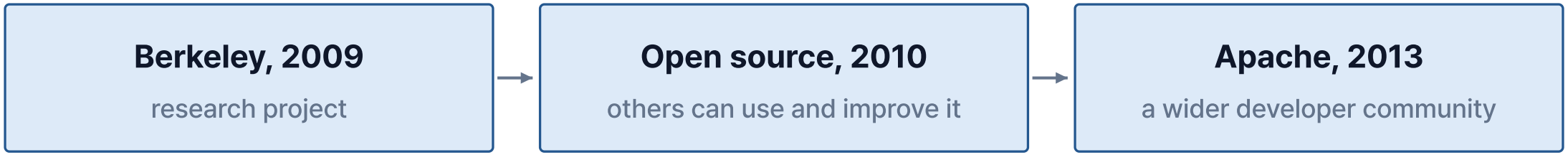
01	MapReduce	split data, map records and combine by key
02	**Spark**	describe table operations and run distributed jobs

§ Describing the Job



Spark Begins at Berkeley

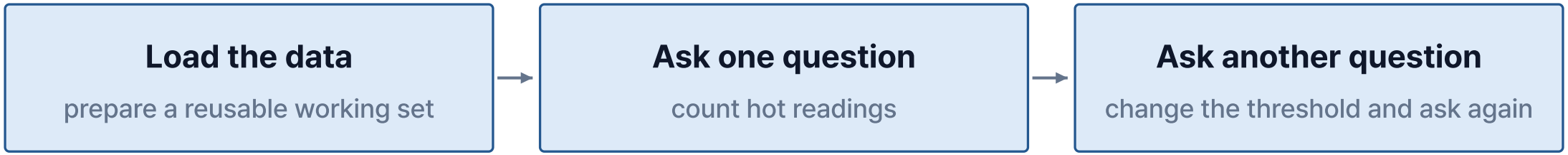
Researchers wanted to reuse the same data across repeated calculations and interactive questions



Spark grew from a university research project into an open-source community

The Same Data, Another Question

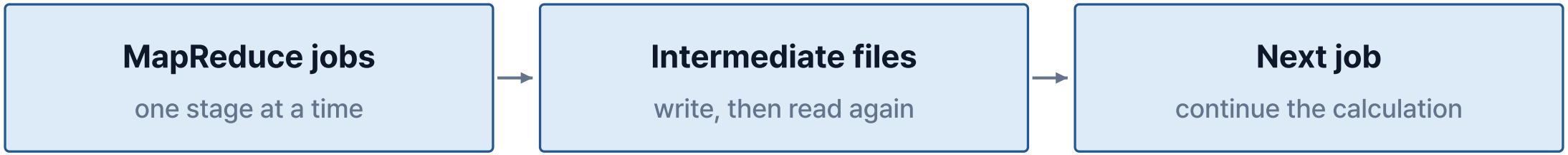
An analyst often changes a question while continuing to use the same dataset



Reusing prepared data can avoid repeating the same preparation for each question

From MapReduce to Spark

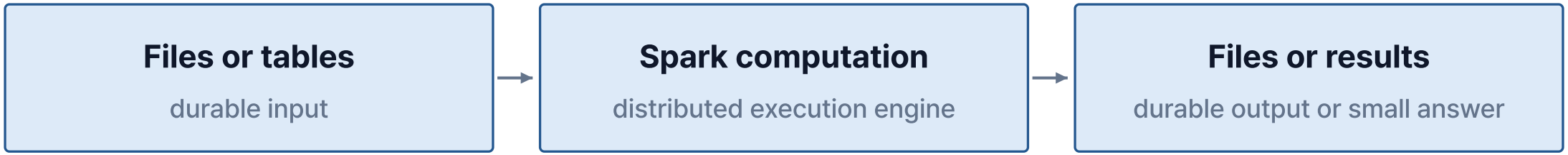
Spark lets us describe several table operations together in one execution plan



Multi-step work motivates an engine that connects operations in one plan

Spark

Spark runs table operations across data pieces; it handles the distributed work



The storage keeps the data; Spark executes the work

A DataFrame: One Table, Many Pieces

events is our input DataFrame: one table whose rows can be stored in different pieces

Partition 1

id	machine	temp
1	M1	70
2	M2	90
3	M1	85

3 rows

Partition 2

id	machine	temp
4	M1	95
5	M3	75
6	M2	60

3 rows

Partition 3

id	machine	temp
7	M3	88
8	M1	82
9	M2	78

3 rows

One events DataFrame, three illustrated input partitions

Start a Spark Session

PySpark is the Python interface to Spark; spark is our entry point for reading data

```
from pyspark.sql import SparkSession

# Create or reuse the Spark entry point named spark
spark = (SparkSession.builder
    .appName("hot-readings")
    .getOrCreate())
```

From a CSV File to a Table

A comma-separated values (CSV) file stores one table row per line

readings.csv: values separated by commas

```
id,machine,temp
1,M1,70
2,M2,90
3,M1,85
```

events: the same values as table rows

id	machine	temp
1	M1	70
2	M2	90
3	M1	85

First line: column names (header)

Schema: column names + value types

The header names columns; the schema gives those columns their value types

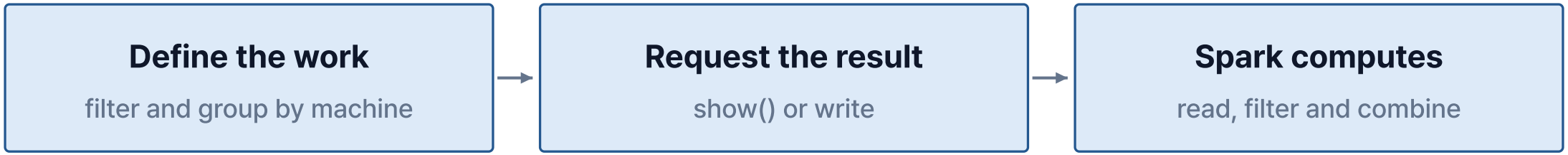
Read Files with a Schema

A schema defines the columns: id is a whole number, machine is text, temp is numeric

```
events = (spark.read
    .schema("id INT, machine STRING, temp DOUBLE")
    .option("header", True)  # First CSV row contains column names
    .csv("raw/readings/2026-11-05/"))  # Folder for one day
```

Describing and Running Work

Lazy evaluation delays result computation until an action requests it



A transformation defines a DataFrame; an action returns or saves a result

Filtering a DataFrame

filter keeps matching rows; hot names that DataFrame; show() requests its rows

```
hot = events.filter(events.temp >= 80)
hot.show()  # Request the rows
```

id	machine	temp
2	M2	90
3	M1	85
4	M1	95
7	M3	88
8	M1	82

Result: five individual readings

The selected rows keep their original id, machine and temperature columns

Counting Groups in a DataFrame

groupBy chooses the grouping column; count defines one row count for each group

```
# Group rows by machine, then count each group
hot_counts = hot.groupBy("machine").count()
hot_counts.show()  # Request the grouped rows
```

machine	count
M1	3
M2	1
M3	1

Result: one count per machine

hot_counts describes three grouped rows rather than five individual readings

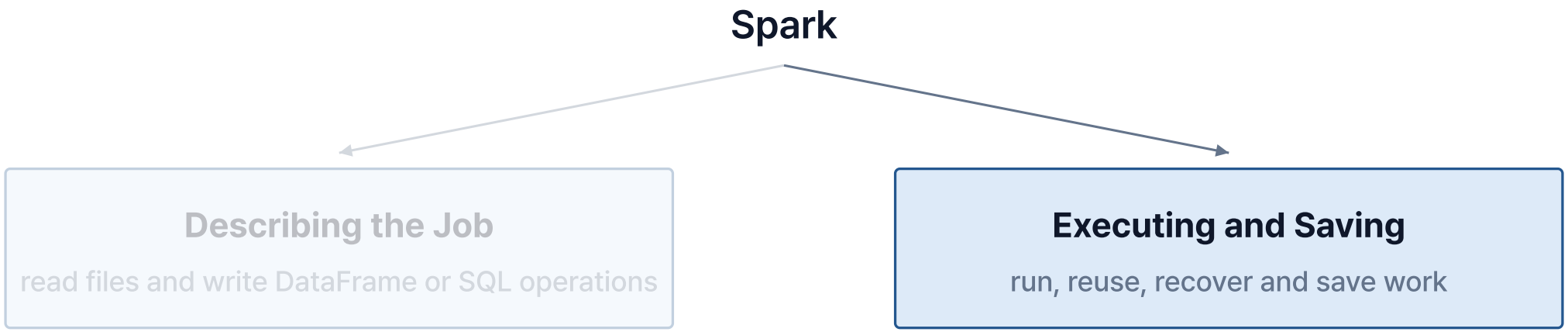
The Same Question in SQL

WHERE keeps the hot rows; GROUP BY machine and COUNT(*) count them per machine

```
events.createOrReplaceTempView("events")  # Name the table for SQL

hot_counts = spark.sql("""
    SELECT machine, COUNT(*) AS count
    FROM events
    WHERE temp >= 80
    GROUP BY machine
""")
hot_counts.show()  # Request the same three grouped rows
```

§ Executing and Saving



Two Meanings of count()

The object before count() determines whether it returns a number or defines grouped rows

Count all hot readings

hot.count()

action: returns one Python integer

5

Count within each machine

hot.groupBy("machine").count()

transformation: defines a DataFrame

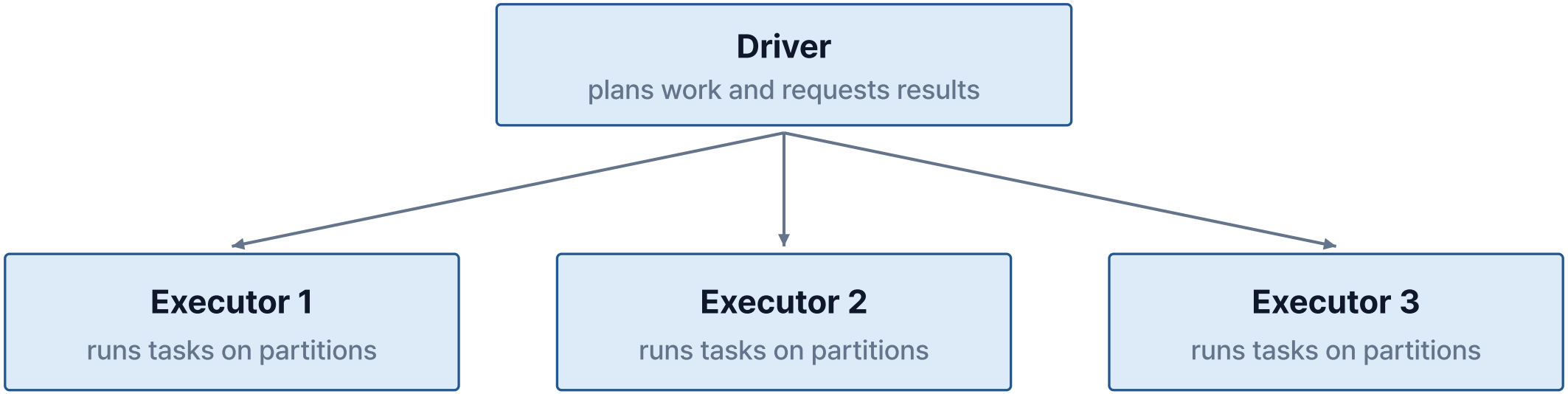
machine	count
M1	3
M2	1
M3	1

Call .show() to request these rows

Five hot readings and three grouped result rows answer different questions

Driver and Executors

Spark calls its coordinator the driver and its worker processes executors



An executor is a process; one machine can host several tasks

One program can coordinate work across several executor processes

A Grouped Count Needs a Merge

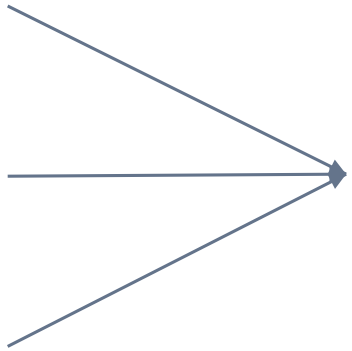
Spark can combine local partial counts before moving them by key

Local answers

P1: M1=1, M2=1

P2: M1=1

P3: M1=1, M3=1



Global answer

machine	count
M1	3
M2	1
M3	1

M1: 1 + 1 + 1 = 3

Local aggregation can reduce traffic; the global merge still matters

Demo: Change the Query Plan

Browser simulation: change the condition, predict the rows, then run an action

Keep temp >= 80 | Group by machine | Run action | Reset

Input P1 id / machine / temp	Input P2 id / machine / temp	Input P3 id / machine / temp
1 / M1 / 70	4 / M1 / 95	7 / M3 / 88
2 / M2 / 90	5 / M3 / 75	8 / M1 / 82
3 / M1 / 85	6 / M2 / 60	9 / M2 / 78

Plan only

filter → group by machine → count

Condition changed: predict the result, then press Run action. This browser model is not Spark.

The grouping choice changes the meaning and number of output rows

Answer: Change the Query Plan

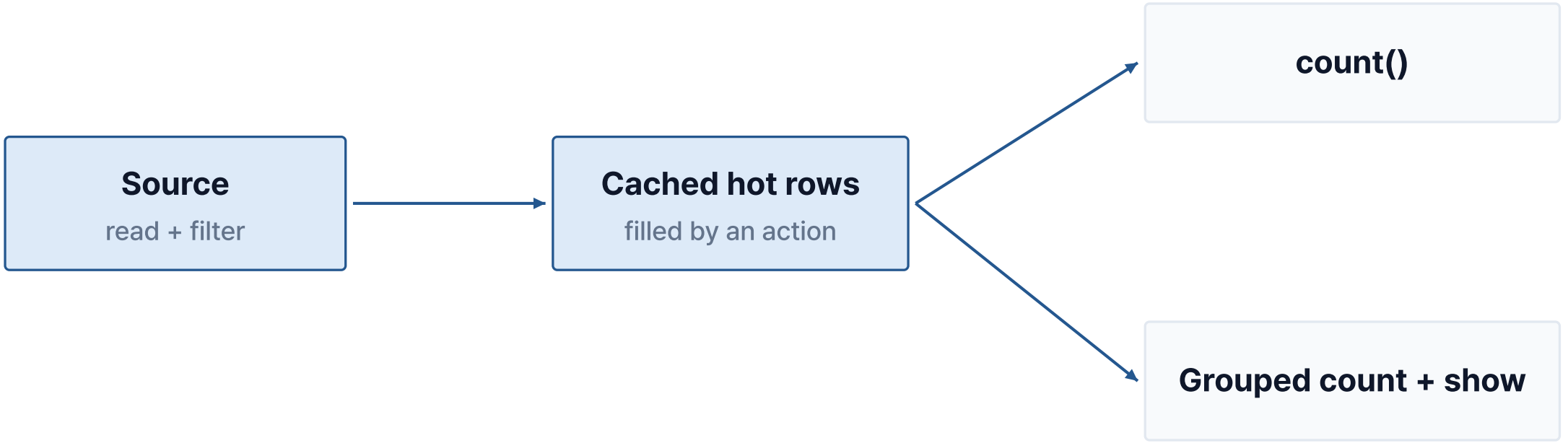
At 85, one M1 reading drops out; the other machines remain

Condition	With grouping	Without grouping
temp >= 80	M1=3, M2=1, M3=1	5 readings
temp >= 85	M1=2, M2=1, M3=1	4 readings
temp >= 100	0 rows	0 rows

Four retained readings produce three grouped rows

Cache Reused Data

When two actions need the same rows, caching keeps computed pieces for reuse



Reuse retained partitions; lost or evicted data can be recomputed

Caching requests reuse; actions populate the cached data

Reuse One Filtered DataFrame

Cache only when repeated reuse justifies its storage cost

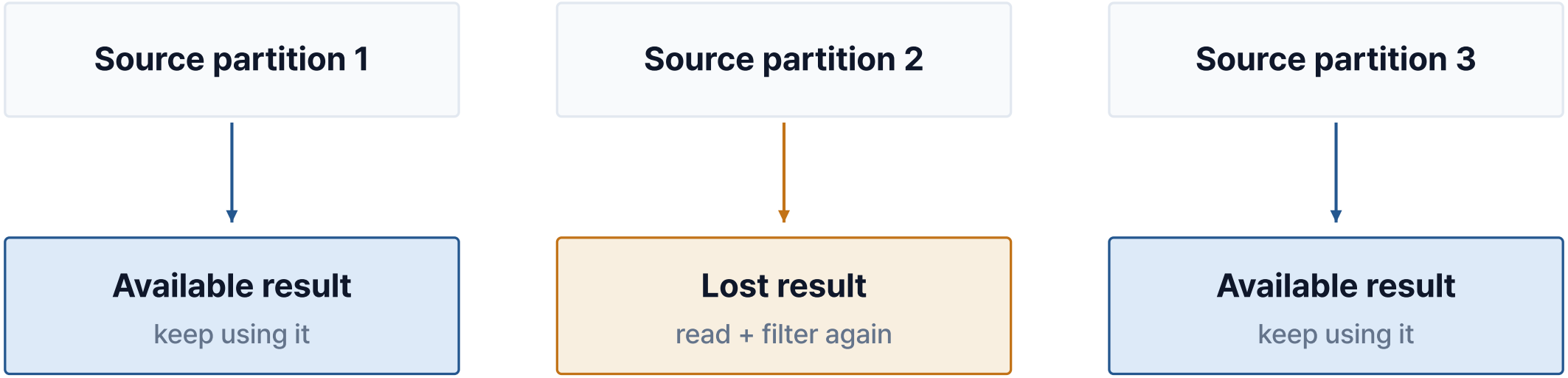
```
hot = events.filter(events.temp >= 80).cache()

hot.count()  # Returns 5; also computes and keeps the hot rows
hot.groupBy("machine").count().show()  # Reuse kept rows

hot.unpersist()  # Release the cache when finished
```

Recover Missing Work

Lineage is the recorded path from input to result, used to recompute lost work



Lineage = dependencies and operations used to rebuild missing data

Independent filters rebuild the lost partition; dependencies determine wider retries

Caveat: Uneven Work

Uneven data creates uneven work; this imbalance is called data skew

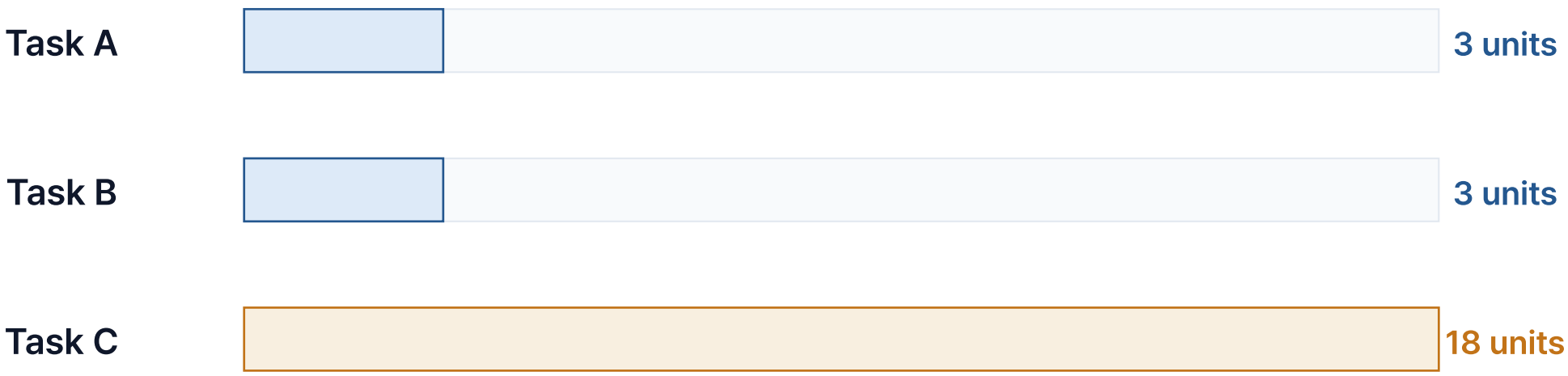
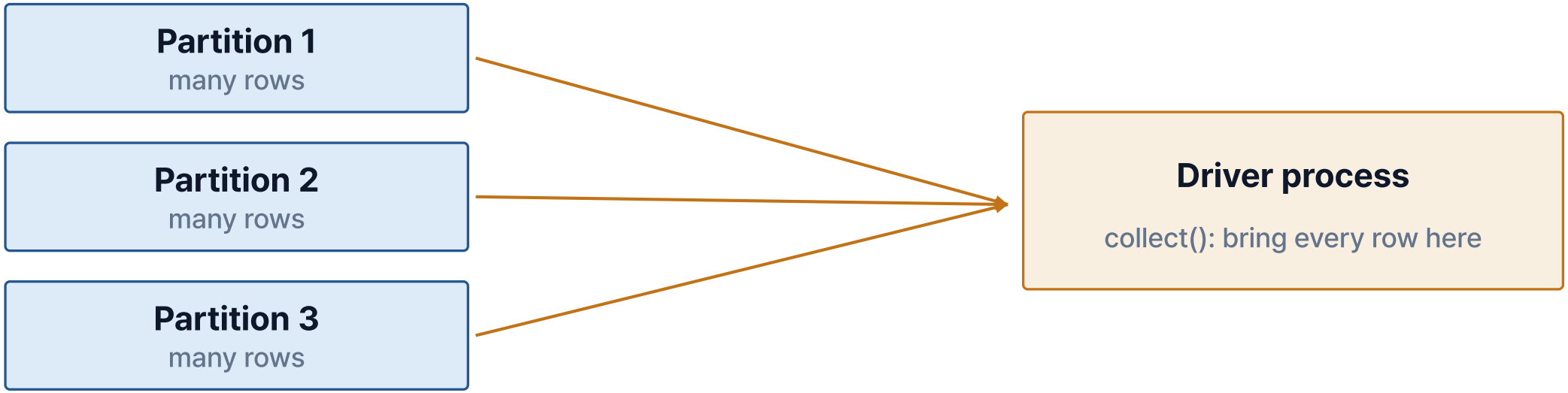


Illustration: equal cost per unit; the job waits for Task C

More workers alone do not remove a heavily loaded key

Caveat: Collecting Too Much

collect() gathers every result row into the driver; a large result may not fit



A distributed input can become a single-machine memory problem

Check the result size before turning distributed data into a local object

Write Distributed Results

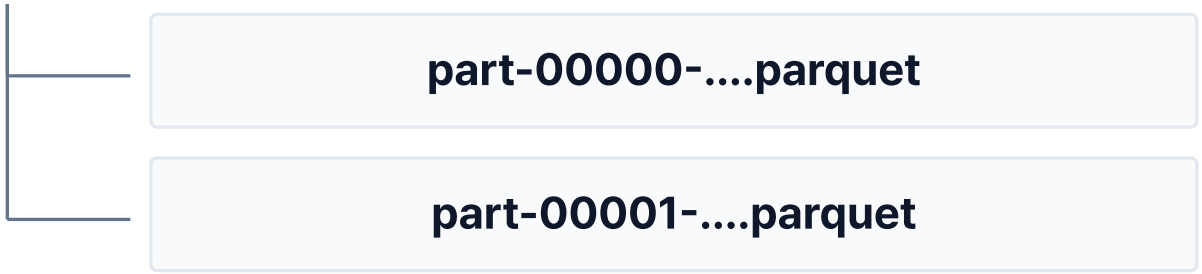
write saves the table; Parquet is a column-based file format for the saved data

```
(hot_counts.write
    .mode("errorifexists")  # Stop if the path exists
    .parquet("output/hot_counts/run_2026_11_05/"))
```

A Dataset Is Often a Folder

Different data pieces can be saved as separate files and read together as one table

hot_counts/run_2026_11_05/



Output is a folder

with data files

Read the folder as a table

Several part files can represent one logical result table

Caveat: Overwriting the Wrong Path

Writing one day of results to the whole history folder can replace other days



Writing one day to the whole path can remove other days

A one-day input does not make a whole-path overwrite safe

One Complete Spark Job

Define a bounded input, transform it, then save the result once

```
events = (spark.read.schema("id INT, machine STRING, temp DOUBLE")
    .option("header", True).csv("raw/readings/2026-11-05/"))

hot_counts = (events.filter(events.temp >= 80)
    .groupBy("machine").count())

(hot_counts.write.mode("errorifexists")
    .parquet("output/hot_counts/run_2026_11_05/"))
```

Takeaway: Big Data I

1. A **partition** is one piece of the input; each piece can be filtered independently
2. A **shuffle** brings matching keys together so their partial answers can be combined
3. Describe work with **DataFrames or SQL**; request results, reuse or recover work, then **save**